EE263 Homework 4

Summer 2026

1. Orthogonal matrices.

a) Show that if U and V are orthogonal, then so is UV.

b) Show that if U is orthogonal, then so is U^{-1}.

c) Suppose that $U \in \mathbb{R}^{2\times 2}$ is orthogonal. Show that U is either a rotation or a reflection. Make clear how you decide whether a given orthogonal U is a rotation or reflection.

2. Groups of equivalent statements. In the list below there are 11 statements about two square matrices A and B in $\mathbb{R}^{n\times n}$.

a) $\mathrm{range}(B) \subseteq \mathrm{range}(A)$.

b) there exists a matrix $Y \in \mathbb{R}^{n\times n}$ such that $B = YA$.

c) $AB = 0$.

d) $BA = 0$.

e) $\mathrm{rank}(\begin{bmatrix} A & B \end{bmatrix}) = \mathrm{rank}(A)$.

f) $\mathrm{range}(A) \perp \mathrm{null}(B^{\mathsf{T}})$.

g) $\mathrm{rank}(\begin{bmatrix} A \\ B \end{bmatrix}) = \mathrm{rank}(A)$.

h) $\mathrm{range}(A) \subseteq \mathrm{null}(B)$.

i) there exists a matrix $Z \in \mathbb{R}^{n\times n}$ such that $B = AZ$.

j) $\mathrm{rank}(\begin{bmatrix} A & B \end{bmatrix}) = \mathrm{rank}(B)$.

k) $\mathrm{null}(A) \subseteq \mathrm{null}(B)$.

Your job is to collect them into (the largest possible) groups of equivalent statements. Two statements are equivalent if each one implies the other. For example, the statement 'A is onto' is equivalent to '$\mathrm{null}(A) = \{0\}$' (when A is square, which we assume here), because every square matrix that is onto has zero nullspace, and vice versa. Two statements are not equivalent if there exist (real) square matrices A and B for which one holds, but the other does not. A group of statements is equivalent if any pair of statements in the group is equivalent.

We want *just* your answer, which will consist of lists of mutually equivalent statements; we do not need any justification.

Put your answer in the following specific form. List each group of equivalent statements on a line, in (alphabetic) order. Each new line should start with the first letter not listed above.

For example, you might give your answer as

a, c, d, h

b, i

e

f, g, j, k.

This means you believe that statements a, c, d, and h are equivalent; statements b and i are equivalent; and statements f, g, j, and k are equivalent. You also believe that the first group of statements is not equivalent to the second, or the third, and so on.

3. (Optional) Complex linear algebra and least-squares. Most of the linear algebra you have seen is unchanged when the scalars, matrices, and vectors are complex, *i.e.*, have complex entries. For example, we say a set of complex vectors $\{v_1, \ldots, v_n\}$ is dependent if there exist complex scalars $\alpha_1, \ldots, \alpha_n$, not all zero, such that $\alpha_1 v_1 + \cdots + \alpha_n v_n = 0$. There are some slight differences when it comes to the inner product and other expressions that, in the real case, involve the transpose operator. For complex matrices (or vectors) we define the *Hermitian conjugate* as the complex conjugate of the transpose. We denote this as A^*, which is equal to $(\overline{A})^\mathsf{T}$. Thus, the ij entry of the matrix A^* is given by $\overline{(A_{ji})}$. The Hermitian conjugate of a matrix is sometimes called its *conjugate transpose* (which is a nice, explanatory name). Note that for a real matrix or vector, the Hermitian conjugate is the same as the transpose. We define the inner product of two complex vectors $u,\ v \in \mathbb{C}^n$ as

$$\langle u, v \rangle = u^* v,$$

which, in general, is a complex number. The norm of a complex vector is defined as

$$\|u\| = \sqrt{\langle u, u \rangle} = \left(|u_1|^2 + \cdots + |u_n|^2\right)^{1/2}.$$

Note that these two expressions agree with the definitions you already know when the vectors are real. The complex least-squares problem is to find the $x \in \mathbb{C}^n$ that minimizes $\|Ax - y\|^2$, where $A \in \mathbb{C}^{m \times n}$ and $y \in \mathbb{C}^m$ are given. Assuming A is full rank and skinny, the solution is $x_{\text{ls}} = A^\dagger y$, where $A^\dagger$ is the (complex) pseudo-inverse of A, given by

$$A^\dagger = (A^* A)^{-1} A^*.$$

(Which also reduces to the pseudo-inverse you've already seen when A is real). There are two general approaches to dealing with complex linear algebra problems. In the first, you simply generalize all the results to work for complex matrices, vectors, and scalars. Another approach is to represent complex matrices and vectors using real matrices and vectors of twice the dimensions, and then you apply what you already know about real linear algebra. We'll explore that idea in this problem. We associate with a complex vector $u \in \mathbb{C}^n$ a *real* vector $\tilde{u} \in \mathbb{R}^{2n}$, given by

$$\tilde{u} = \begin{bmatrix} \Re u \\ \Im u \end{bmatrix}.$$

We associate with a complex matrix $A \in \mathbb{C}^{m \times n}$ the *real* matrix $\tilde{A} \in \mathbb{R}^{2m \times 2n}$ given by

$$\tilde{A} = \begin{bmatrix} \Re A & -\Im A \\ \Im A & \Re A \end{bmatrix}.$$

a) What is the relation between $\langle u, v\rangle$ and $\langle \tilde{u}, \tilde{v}\rangle$? Note that the first inner product involves complex vectors and the second involves real vectors.

b) What is the relation between $\|u\|$ and $\|\tilde{u}\|$?

c) What is the relation between Au (complex matrix-vector multiplication) and $\tilde{A}\tilde{u}$ (real matrix-vector multiplication)?

d) What is the relation between $\tilde{A}^{\mathsf{T}}$ and A^*?

e) Using the results above, verify that $A^\dagger y$ solves the complex least-squares problem of minimizing $\|Ax - y\|$ (where A, x, y are complex). Express $A^\dagger y$ in terms of the real and imaginary parts of A and y. (You don't need to simplify your expression; you can leave block matrices in it.)

4. (Optional) Choosing a basis to make interpolation easy. In this question, we consider the situation where we have discrete-time signals, which we view as vectors in $\mathbb{R}^n$. We have samples of a signal y at particular times, and would like to fill in the rest of the signal using interpolation. For example, if we have m sample times $t_1, t_2, \ldots, t_m$, we will denote the corresponding sample values by y_{t_i} for $i = 1, \ldots, m$.

a) We first consider the case of *linear interpolation*. Here we have m sample times, with the additional restriction that $t_1 = 1$ and $t_m = n$, so that the first sample is the first value of the signal, and the last sample is the last value.

Between any two samples y_{t_j} and $y_{t_{j+1}}$, we construct y by filling in the missing values using linear interpolation. That is, if $t_j \le t \le t_{j+1}$, we set

$$y_t = (1 - \alpha_t) y_{t_j} + \alpha_t y_{t_{j+1}}$$

where

$$\alpha_t = \frac{t - t_j}{t_{j+1} - t_j}$$

We are given the values at $t_1, \ldots, t_m$. Consider the set of all signals y which are constructible by linear interpolation between the values at $t_1, \ldots, t_m$. Show that this set is a subspace.

b) Let $x \in \mathbb{R}^m$ be the vector of sample values

$$x = \begin{bmatrix} y_{t_1} \\ y_{t_2} \\ \vdots \\ y_{t_m} \end{bmatrix}$$

Suppose $n = 20$, $m = 4$, and

$$t_1 = 1 \quad t_2 = 5 \quad t_3 = 11 \quad t_4 = 20$$

Construct the matrix A such that $y = Ax$. Print it out. Plot the m columns of A. For each column, the index of each element should appear on the x axis and the value of each element should appear on the y axis.

c) What is the rank of A. Explain why.

d) Suppose $y = Ax$ for some x, and you are given y and A. One algorithm for finding x is to use the pseudoinverse

$$x = A^\dagger y$$

In this case, there is a much simpler algorithm. What is it?

e) The matrix A is quite magical, in that it gives a very simple relationship between x and y, because y is constructed by interpolation. Specifically

$$y = a_1 y_{t_1} + a_2 y_{t_2} + \cdots + a_m y_{t_m}$$

where the a_i are the columns of A. In other words, to construct the entire signal y, use the values at the sampling times t_i as coefficients in a linear combination of the basis functions.

In fact, any subspace $S \subset \mathbb{R}^n$ can be constructed in this way, if one chooses the sample points $t_1, \ldots, t_m$ and vectors $a_1, \ldots, a_m$ carefully. We will see how to do this. We start with a matrix $B \in \mathbb{R}^{n \times m}$ whose columns are a basis for the subspace. Write B in terms of its rows, as

$$B = \begin{bmatrix} b_1^\mathsf{T} \\ b_2^\mathsf{T} \\ \vdots \\ b_n^\mathsf{T} \end{bmatrix}$$

Show that there exists $t_1, \ldots, t_m$ such that the corresponding submatrix

$$Z = \begin{bmatrix} b_{t_1}^\mathsf{T} \\ b_{t_2}^\mathsf{T} \\ \vdots \\ b_{t_m}^\mathsf{T} \end{bmatrix}$$

is invertible. Give an algorithm for finding such $t_1, \ldots, t_m$. (You can turn it in the the form of printed out code, or in written-out pseudo-code.)

f) Given the specially chosen sampling times $t_1, \ldots, t_m$ from the previous part, we can now choose our special basis $v^{(1)}, \ldots, v^{(m)} \in \mathbb{R}^n$ for S. We would like these vectors to satisfy

$$y = v^{(1)} y_{t_1} + v^{(2)} y_{t_2} + \cdots + v^{(m)} y_{t_m} \tag{1}$$

for any $y \in \operatorname{range} B$, which is the 'nice' interpolation property. (It means that we can use the values of y at the sampling times t_i as the coefficients of the basis functions.)

Show that if $v^{(1)}, \ldots, v^{(m)} \in \mathbb{R}^n$ is a basis for a subspace S, and equation (1) holds for all $y \in S$ then

$$v_{t_j}^{(i)} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

g) Show the converse; that is, if

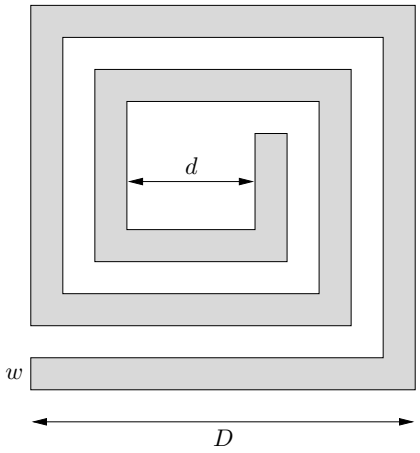
$$v_{t_j}^{(i)} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

then equation (1) holds for all $y \in S$.

h) You are given a matrix $B \in \mathbb{R}^{n \times m}$ with $\text{null}(B) = \{0\}$. Give an algorithm for finding a basis $v_1, \ldots, v_m$ for $\text{range}(B)$ satisfying equation (1).

i) In the file `interp.json` you will find an $n \times m$ matrix B, whose columns span a subspace S. Apply your algorithm from the previous parts to find $t_1, \ldots, t_m$ and $v^{(1)}, \ldots, v^{(m)}$. What are $t_1, \ldots, t_m$? Plot the basis vectors $v^{(1)}, \ldots, v^{(m)}$.

5. Approximate inductance formula. The figure below shows a *planar spiral inductor*, implemented in CMOS, for use in RF circuits.



The inductor is characterized by four key parameters:

- n, the number of turns (which is a multiple of 1/4, but that needn't concern us)
- w, the width of the wire
- d, the inner diameter
- D, the outer diameter

The inductance L of such an inductor is a complicated function of the parameters n, w, d, and D. The inductance L can be found by solving Maxwell's equations, which takes considerable computer time, or by fabricating the inductor and measuring the inductance. In this problem you will develop a simple approximate inductance model of the form

$$\hat{L} = \alpha n^{\beta_1} w^{\beta_2} d^{\beta_3} D^{\beta_4},$$

where $\alpha, \beta_1, \beta_2, \beta_3, \beta_4 \in \mathbb{R}$ are constants that characterize the approximate model. (since L is positive, we have $\alpha > 0$, but the constants $\beta_2, \ldots, \beta_4$ can be negative.) This simple approximate model, if accurate enough, can be used for design of planar spiral inductors.

The file `inductor_data.json` on the course web site contains data for 50 inductors. (The data is real, not that it would affect how you solve the problem ...) For inductor i, we give parameters n_i, w_i, d_i, and D_i (all in μm), and also, the inductance L_i (in nH) obtained from measurements. (The data are organized as vectors of length 50. Thus, for example, w_{13} gives the wire width of inductor 13.) Your task, *i.e.*, the problem, is to find α, $\beta_1, \ldots, \beta_4$ so that

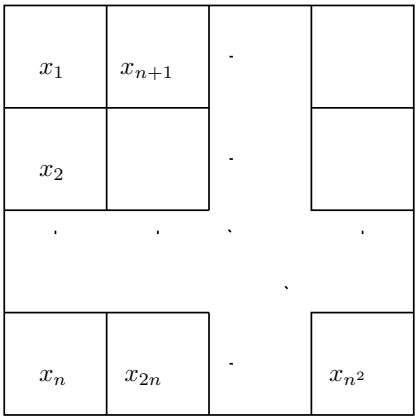
$$\hat{L}_i = \alpha n_i^{\beta_1} w_i^{\beta_2} d_i^{\beta_3} D_i^{\beta_4} \approx L_i \qquad \text{for } i = 1, \ldots, 50.$$

Your solution must include a clear description of how you found your parameters, as well as their actual numerical values. Note that we have not specified the criterion that you use to judge the approximate model (*i.e.*, the fit between $\hat{L}_i$ and L_i); we leave that to your engineering judgment. But be sure to tell us what criterion you use. We define the *percentage error* between $\hat{L}_i$ and L_i as

$$e_i = 100|\hat{L}_i - L_i|/L_i.$$

Find the average percentage error for your model, *i.e.*, $(e_1 + \cdots + e_{50})/50$. (We are only asking you to find the average percentage error for your model; we do not require that your model minimize the average percentage error.) *Hint*: you might find it easier to work with $\log L$.

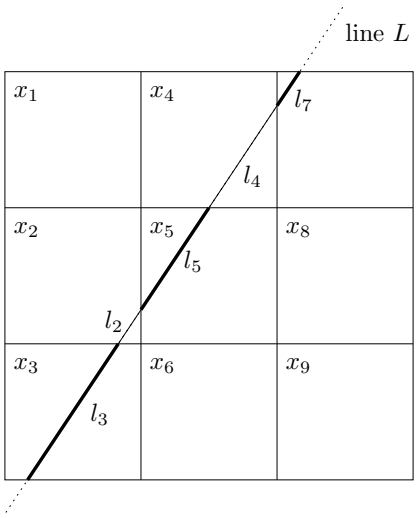
6. Image reconstruction from line integrals. In this problem we explore a simple version of a tomography problem. We consider a square region, which we divide into an $n \times n$ array of square pixels, as shown below.



The pixels are indexed column first, by a single index i ranging from 1 to n^2, as shown above. We are interested in some physical property such as density (say) which varies over the region. To simplify things, we'll assume that the density is constant inside each pixel, and we denote by x_i the density in pixel i, $i = 1, \ldots, n^2$. Thus, $x \in \mathbb{R}^{n^2}$ is a vector that describes the density across the rectangular array of pixels. The problem is to estimate the vector of densities x, from a set of sensor measurements that we now describe. Each sensor measurement is a *line integral* of the density over a line L. In addition, each measurement is corrupted by a (small) noise term. In other words, the sensor measurement for line L is given by

$$\sum_{i=1}^{n^2} l_i x_i + v,$$

where l_i is the length of the intersection of line L with pixel i (or zero if they don't intersect), and v is a (small) measurement noise. This is illustrated below for a problem with $n = 3$. In this example, we have $l_1 = l_6 = l_8 = l_9 = 0$.



Now suppose we have N line integral measurements, associated with lines $L_1, \ldots, L_N$. From these measurements, we want to estimate the vector of densities x. The lines are characterized by the intersection lengths

$$l_{ij}, \quad i = 1, \ldots, n^2, \quad j = 1, \ldots, N,$$

where l_{ij} gives the length of the intersection of line L_j with pixel i. Then, the whole set of measurements forms a vector $y \in \mathbb{R}^N$ whose elements are given by

$$y_j = \sum_{i=1}^{n^2} l_{ij} x_i + v_j, \quad j = 1, \ldots, N.$$

And now the problem: you will reconstruct the pixel densities x from the line integral measurements y. The class webpage contains the file `tomo_data.json`, which contains the following variables:

- `N`, the number of measurements (N),
- `npixels`, the side length in pixels of the square region (n),
- `y`, a vector with the line integrals y_j, $j = 1, \ldots, N$,
- `line_pixel_lengths`, an $n^2 \times N$ matrix containing the intersection lengths l_{ij} of each pixel $i = 1, \ldots, n^2$ (ordered column-first as in the above diagram) and each line $j = 1, \ldots, N$,
- `lines_d`, a vector containing the displacement (distance from the center of the region in pixel lengths) d_j of each line $j = 1, \ldots, N$, and

- `lines_theta`, a vector containing the angles θ_j of each line $j = 1, \ldots, N$.

(You shouldn't need `lines_d` or `lines_theta`, but we're providing them to give you some idea of how the data was generated. Similarly, the file `tmeasure.jl` shows how we computed the measurements, but you don't need it or anything in it to solve the problem. The variable `line_pixel_lengths` was computed using the function in this file.)

Use this information to find x, and display it as an image (of n by n pixels). You'll know you have it right.

Julia hints:

- The `reshape` function might help with converting between vectors and matrices, for example, `A = reshape(v, m, n)` will convert a vector with $v = mn$ elements into an $m \times n$ matrix.

- To display a matrix `A` as a grayscale image, you can use: (or any method that works for you)

```
heatmap(A, yflip=true, aspect_ratio=:equal, color=:gist_gray,
        cbar=:none, framestyle=:none)
```

  You'll need to have loaded the JuliaPlots package with `using Plots` to access the `heatmap` function. (The `yflip` argument gets it to plot the origin in the top-left rather than the bottom-left.)

Note: While irrelevant to your solution, this is actually a simple version of *tomography*, best known for its application in medical imaging as the CAT scan. If an *x-ray* gets attenuated at rate x_i in pixel i (a little piece of a cross-section of your body), the j-th measurement is

$$z_j = \prod_{i=1}^{n^2} e^{-x_i l_{ij}},$$

with the l_{ij} as before. Now define $y_j = -\log z_j$, and we get

$$y_j = \sum_{i=1}^{n^2} x_i l_{ij}.$$

7. Curve-smoothing. We are given a function $F : [0, 1] \to \mathbb{R}$ (whose graph gives a curve in $\mathbb{R}^2$). Our goal is to find another function $G : [0, 1] \to \mathbb{R}$, which is a *smoothed* version of F. We'll judge the smoothed version G of F in two ways:

- *Mean-square deviation from F,* defined as

$$D = \int_0^1 (F(t) - G(t))^2 \, dt.$$

- *Mean-square curvature,* defined as

$$C = \int_0^1 G''(t)^2 \, dt.$$

We want *both* D and C to be small, so we have a problem with two objectives. In general there will be a trade-off between the two objectives. At one extreme, we can choose $G = F$, which makes $D = 0$; at the other extreme, we can choose G to be an affine function (*i.e.*, to have $G''(t) = 0$ for all $t \in [0, 1]$), in which case $C = 0$. The problem is to identify the optimal trade-off curve between C and D, and explain how to find smoothed functions G on the optimal trade-off curve. To reduce the problem to a finite-dimensional one, we will represent the functions F and G (approximately) by vectors $f,\ g \in \mathbb{R}^n$, where

$$f_i = F(i/n), \quad g_i = G(i/n).$$

You can assume that n is chosen large enough to represent the functions well. Using this representation we will use the following objectives, which approximate the ones defined for the functions above:

- *Mean-square deviation,* defined as

$$d = \frac{1}{n}\sum_{i=1}^{n}(f_i - g_i)^2.$$

- *Mean-square curvature,* defined as

$$c = \frac{1}{n-2}\sum_{i=2}^{n-1}\left(\frac{g_{i+1} - 2g_i + g_{i-1}}{1/n^2}\right)^2.$$

In our definition of c, note that

$$\frac{g_{i+1} - 2g_i + g_{i-1}}{1/n^2}$$

gives a simple approximation of $G''(i/n)$. You will only work with this approximate version of the problem, *i.e.*, the vectors f and g and the objectives c and d.

a) Explain how to find g that minimizes $d + \mu c$, where $\mu \geq 0$ is a parameter that gives the relative weighting of sum-square curvature compared to sum-square deviation. Does your method always work? If there are some assumptions you need to make (say, on rank of some matrix, independence of some vectors, etc.), state them clearly. Explain how to obtain the two extreme cases: $\mu = 0$, which corresponds to minimizing d without regard for c, and also the solution obtained as $\mu \to \infty$ (*i.e.*, as we put more and more weight on minimizing curvature).

b) Get the file `curve_smoothing.json` from the course web site. This file defines a specific vector f that you will use. Find and plot the optimal trade-off curve between d and c. Be sure to identify any critical points (such as, for example, any intersection of the curve with an axis). Plot the optimal g for the two extreme cases $\mu = 0$ and $\mu \to \infty$, and for three values of μ in between (chosen to show the trade-off nicely). On your plots of g, be sure to include also a plot of f, say with dotted line type, for reference.